THE HUNGRY HELM — VIVA VOCE DEFENSE & DEMONSTRATION GUIDE

Agile Software Development (CSET455) | Evaluator: Rashmi Rani | Student: Prikshit Sharma (S24CSEU1823)

Live Jira Project: *https://prikshittt777.atlassian.net (Key: **THH**, Board ID: **167**, Dashboard ID: **10001**)*
This viva preparation guide equips you to defend every Agile principle and technical decision, linking each concept directly to your live Jira project issues (THH-3 to THH-22) and running codebase.

Part 1: Agile, Scrum & Requirements Engineering (Labs 1 & 2)

Q1: Why Scrum instead of Kanban for The Hungry Helm?

Direct Answer: The Hungry Helm is an iteration-driven product development project with clear time-boxed milestones (Sprints 1 to 4) and sequential technical dependencies (database seed -> auth -> ordering -> KDS). Scrum's sprint cadence and sprint goals provide delivery predictability. Kanban is better suited for continuous support streams like IT helpdesk ticketing (as in Lab 4).

Q2: How do your User Stories meet the INVEST criteria?

Direct Answer: Each of our 14 stories (THH-9 to THH-22) is Independent, Negotiable, Valuable, Estimable, Small, and Testable. For example, THH-16 (Server-Side Price Verification) is Small (5 SP), Valuable to security, and Testable via BDD Given/When/Then criteria: *'Given a cart with tampered item price INR 1, when checkout is submitted, backend fetches DB price INR 180 and persists verified total.'*

Q3: What is the difference between an Epic and a Story in your Jira project?

Direct Answer: In our Jira project, an **Epic** represents a high-level strategic capability spanning multiple iterations (e.g. THH-6: Cart Management & Tamper-Proof Order Placement). A **User Story** is a granular vertical slice sized to fit comfortably in a sprint (e.g. THH-15: Persistent Cart = 5 SP, and THH-16: Server Checkout = 5 SP).

Part 2: Estimation, Velocity Planning & MoSCoW (Labs 3, 5, 6)

Q4: What was your agreed baseline anchor story, and why?

Direct Answer: Our agreed baseline anchor is THH-9: Multi-Outlet Seed & Campus Data Model, sized at **3 Story Points**. It was chosen because seeding 5 campus stalls in Mongoose involves well-understood schema structures, standard JSON fixtures, and zero external integration risk. All other stories were sized relative to this 3-point anchor.

Q5: How did you allocate stories across your 4 Sprints?

Direct Answer: We calibrated a team velocity of **20 Story Points per 2-week Sprint**. Across our 67 total points, we sequenced stories by dependencies: Sprint 140 (18 pts - Foundation), Sprint 141 (18 pts - Ordering), Sprint 142 (18 pts - KDS & Status), and Sprint 143 (13 pts - Hardening). No sprint exceeds the 20-point capacity.

Q6: How did you handle scope trade-offs using MoSCoW when deadlines tightened?

Direct Answer: Rather than using critical-path crashing (unrealistic overtime) or fast-tracking (skipping QA), we applied **MoSCoW prioritization**. When simulating an accelerated deadline, non-essential features (Razorpay online payment, 8 pts, and FCM push notifications, 5 pts) were deferred to v1.1 (Won't Have v1.0), protecting the stability of our core ordering and KDS pipeline.

Part 3: Custom Jira Workflows & Quality Gates (Lab 7)

Q7: What custom workflow rules did you configure on project THH?

Direct Answer: We configured the THH_Software_Scrum_Workflow with 5 statuses: **Backlog -> To Do -> In Progress -> In Review / QA -> Done**, plus a reverse defect loop from In Review back to In Progress.
• **Condition:** Only project members (Prikshit Sharma) can execute status transitions.
• **Validator:** The 'Approve & Done' transition requires the Fix Version (v1.0) and Resolution fields to be non-empty.
• **Post-Function:** Automatically sets Resolution to 'Completed' and logs transition timestamps.

Part 4: Real-World Architecture & Code Verification

Q8: How does your application connect to MongoDB, and how did you solve connection dropouts?

Direct Answer: In backend/server.js, we implemented an architectural fallback using mongodb-memory-server. When the server boots, it first attempts to connect to MongoDB Atlas. If network restrictions, captive portals, or firewall rules block Atlas, the catch block automatically boots an in-memory MongoDB instance and triggers seedDatabase() immediately. Evaluators can run the system anywhere with zero database setup.

Q9: How does the server prevent client-side price tampering during order placement?

Direct Answer: In backend/controllers/orderController.js (linked to story THH-16), the server never trusts client-submitted prices. It takes the item IDs from the cart, queries the database for the canonical item prices, recalculates the subtotal and taxes on the server, and persists the verified amount. If a client intercepts the request and alters INR 180 to INR 1, the server overwrites it with INR 180.

Q10: Why did you shift the backend port from 5000 to 5001?

Direct Answer: On macOS Sonoma and Sequoia, the native ControlCenter AirPlay Receiver service listens on port 5000 by default (EADDRINUSE: :::5000). In story THH-20, we configured backend/.env to PORT=5001 and updated frontend/script.js to target port 5001, eliminating local port collisions.

Part 5: Step-by-Step Live Demonstration Script for Faculty

Step 1: Open Live Jira Workspace
Open https://prikshittt777.atlassian.net/jira/software/c/projects/THH/boards/167/backlog. Point out the 6 Epics (THH-3 to 8), 14 User Stories (THH-9 to 22), and 4 Sprints (140 to 143).

Step 2: Show Active Scrum Board
Click 'Board' (ID: 167). Show Sprint 140 active with THH-9 & THH-10 in Done, THH-11 in Progress, and THH-12 in To Do.

Step 3: Show Custom Dashboard
Navigate to https://prikshittt777.atlassian.net/jira/dashboards/10001. Show the 4 configured gadgets: Sprint Burndown, Created vs Resolved, Filter Stats, and Sprint Health.

Step 4: Demonstrate Running Application
Open http://localhost:3000. Log in as student (prikshit@bennett.edu.in / password). Show campus food chart with live badges. Add Kathi Roll to cart.

Step 5: Demonstrate Staff KDS
Open http://localhost:3000/kds.html in incognito. Log in as staff (IK-STAFF-001 / staff123). Advance order status from incoming -> preparing -> ready. Show student voyage log compass lighting up in real time.